

GRIDS: Interactive Layout Design with Integer Programming

Verified reference summary • Year: 2020 • Source bibliography: docs/REFERENCES.md

What the paper contributes

GRIDS formulates layout design as an integer-programming problem. It exposes packing, alignment, grouping, and preferred-position objectives so designers can combine constraints and let a solver search for valid layouts.

Method / evaluation stance

The paper is not about slide decks specifically, but it is highly relevant to slidekit because it treats layout quality as explicit mathematical structure rather than a visual afterthought. Alignment and grouping become constraints that can be optimized and audited.

How slidekit uses it

slidekit applies the same spirit with a narrower production goal. It does not solve arbitrary user-interface layouts with MILP; instead, each slide component has a deterministic handler that emits bounding boxes. The linter and aesthetic score then evaluate alignment, spacing, and balance over those boxes.

YAML vs HTML takeaway

Compared with HTML/CSS, GRIDS-like constraint thinking fits YAML well. YAML declares intent and content slots; the layout engine owns geometry. HTML can express the final layout directly, but once the DOM and CSS are arbitrary, it is harder to prove that every deck follows the intended design grammar.

Bottom line

The paper supports a pragmatic split: use research-style rendered artifacts when the question depends on final pixels or human preference collection, but keep slidekit production decks in a constrained YAML IR so layout, linting, and agent fixes remain deterministic, inspectable, and cheap.

Primary slidekit mapping	balance / alignment / whitespace / overlap / contrast / richness / consistency, depending on the paper
Operational risk	HTML metrics can drift with rendering environment; YAML metrics can miss pixel-only effects. Use each where it is strongest.